Real Estate AI Platform

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	HARSHIT MITTAL	EN22CS301401
2.	HARSHAL DESHMUKH	EN22CS301396
3.	HARSHIT CHOUDHARY	EN22CS301400
4.	SALONEE CHOUHAN	EN23CS3L1017
5.	HARSHIT SHARMA	EN22CS301403
6.	HARSHIT BHARDWAJ	EN22CS301399

Group Name: Group 11D4

Project Number: DO-47

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025-26

1. Problem Statement & Objectives

1.1 Problem Statement

The real estate market involves vast amounts of unstructured data regarding property listings, pricing, and amenities. Traditional platforms rely on static search filters, which often fail to answer specific, nuanced user queries (e.g., "Which property is best for a small family that needs a home office?"). While Generative AI offers a solution to this, traditional LLMs often suffer from "hallucinations," providing incorrect property details because they lack access to real-time, ground-truth data.

Furthermore, deploying and managing such complex, AI-integrated full-stack applications is traditionally a manual, error-prone, and time-consuming process. There is a critical need for an intelligent system that combines factual, data-grounded AI assistance with a robust, automated DevOps pipeline to ensure seamless scalability and rapid deployment.

1.2 Project Objectives

The main objectives of the AI-Powered Real Estate Platform are:

1. **Develop an Intelligent Assistant:** Implement a Retrieval-Augmented Generation (RAG) pipeline to provide accurate, fact-based responses to user queries using real property data.
2. **Automate Infrastructure:** Use Infrastructure as Code (IaC) and configuration management to automate the provisioning of AWS environments.
3. **Establish CI/CD:** Create an automated continuous integration and deployment pipeline to streamline software releases.
4. **Ensure Scalability:** Containerize the application and utilize orchestration tools to manage traffic and ensure high availability.
5. **Provide Centralized Management:** Deliver an admin dashboard for efficient property listing management and database synchronization.

1.3 Scope of the Project

The scope of this project focuses on the end-to-end development and deployment of a full-stack, cloud-native web application.

The system allows users to securely register, log in, browse property listings, and interact with an AI assistant that provides contextual property recommendations. The backend handles the RAG pipeline, converting property data into vector embeddings for semantic search. The project heavily emphasizes the DevOps lifecycle, incorporating automated AWS EC2 provisioning via Terraform, server configuration via Ansible, containerization with Docker, and automated pipelines with Jenkins.

The scope is limited to the digital platform's architecture, AI integration, and automated deployment processes, and does not include facilitating actual financial transactions or legal real estate documentation.

2. Proposed Solution

2.1 Key Features

The AI-Powered Real Estate Platform provides several intelligent and automated features designed to modernize property search and software delivery:

- **RAG-Based AI Assistant:** Eliminates AI hallucinations by injecting retrieved property data (from a vector database) directly into the LLM context window before generating responses.
- **Semantic Property Search:** Allows users to find homes using natural language descriptions rather than rigid dropdown filters.
- **Automated CI/CD Pipeline:** Uses Jenkins to automatically build and push Docker images to DockerHub upon new code commits.
- **Infrastructure as Code (IaC):** Utilizes Terraform to automatically provision AWS EC2 instances, security groups, and networking rules.
- **Configuration Management:** Employs Ansible to automate the installation of dependencies (like Docker) on remote servers without manual SSH intervention.
- **Containerized Microservices:** Isolates the frontend and backend environments using Docker, orchestrated by Kubernetes for high availability.
- **Admin Property Management:** A secure interface for administrators to perform CRUD operations on property listings.

2.2 Overall Architecture / Workflow

The system follows a containerized microservices architecture with a fully integrated DevOps pipeline.

Application Flow: The frontend is developed using React with Vite, providing a fast UI for property browsing and AI chat. User requests are routed to a FastAPI backend running on port 8000. For AI queries, the backend uses Sentence Transformers to convert the user's text into embeddings, queries ChromaDB for similar property vectors, and sends the combined context to the Google Gemini LLM via LangChain/CrewAI to formulate a response. Transactional data (users, property details) is stored in PostgreSQL (Neon DB).

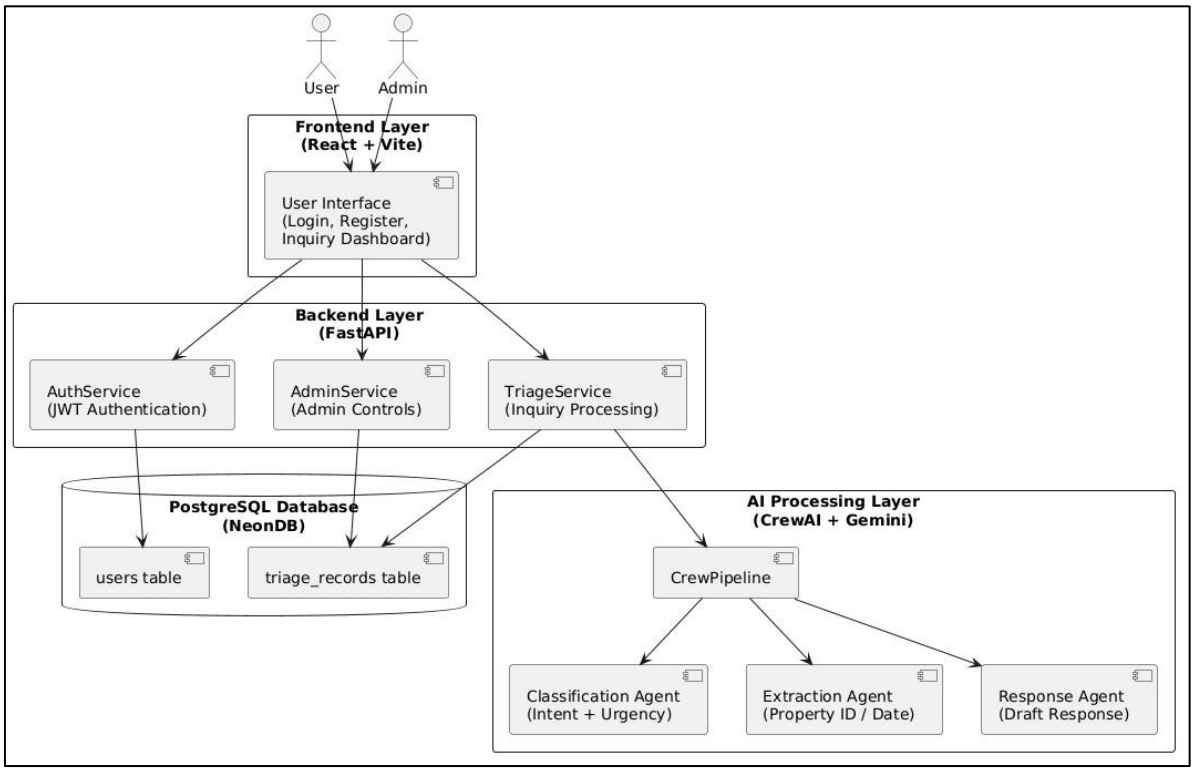
DevOps Workflow: A developer pushes code to the GitHub repository, which triggers a Jenkins CI/CD pipeline. Jenkins builds the Docker images (`realestate-frontend` and `realestate-backend`) and pushes them to DockerHub. Simultaneously, Terraform provisions an AWS EC2 instance, and Ansible configures the server environment. Finally, Kubernetes pulls the updated images from DockerHub and deploys the containers into

isolated Pods, making the application live and accessible to the internet through configured AWS Security Groups.

2.3 Tools & Technologies Used

Category	Technology / Tool	Purpose
Frontend Framework	React (with Vite)	Builds the user interface for property listings and AI chat.
Backend Framework	FastAPI	Handles REST APIs, authentication, and application logic.
AI Orchestration	LangChain & CrewAI	Manages the RAG pipeline and multi-agent workflows.
Large Language Model	Google Gemini	Generates conversational responses based on retrieved context.
Relational Database	PostgreSQL (NeonDB)	Stores structured user accounts and property listings.
Vector Database	ChromaDB	Stores property embeddings for semantic AI search.
Containerization	Docker & DockerHub	Packages applications and acts as an image registry.
Infrastructure as Code	Terraform	Automates the provisioning of AWS EC2 and networking.

Category	Technology / Tool	Purpose
Configuration Mgmt	Ansible	Automates software installation on remote cloud servers.
CI/CD Pipeline	Github Actions	Automates code building, testing, and deployment.
Orchestration	Kubernetes	Manages container scaling, load balancing, and networking.



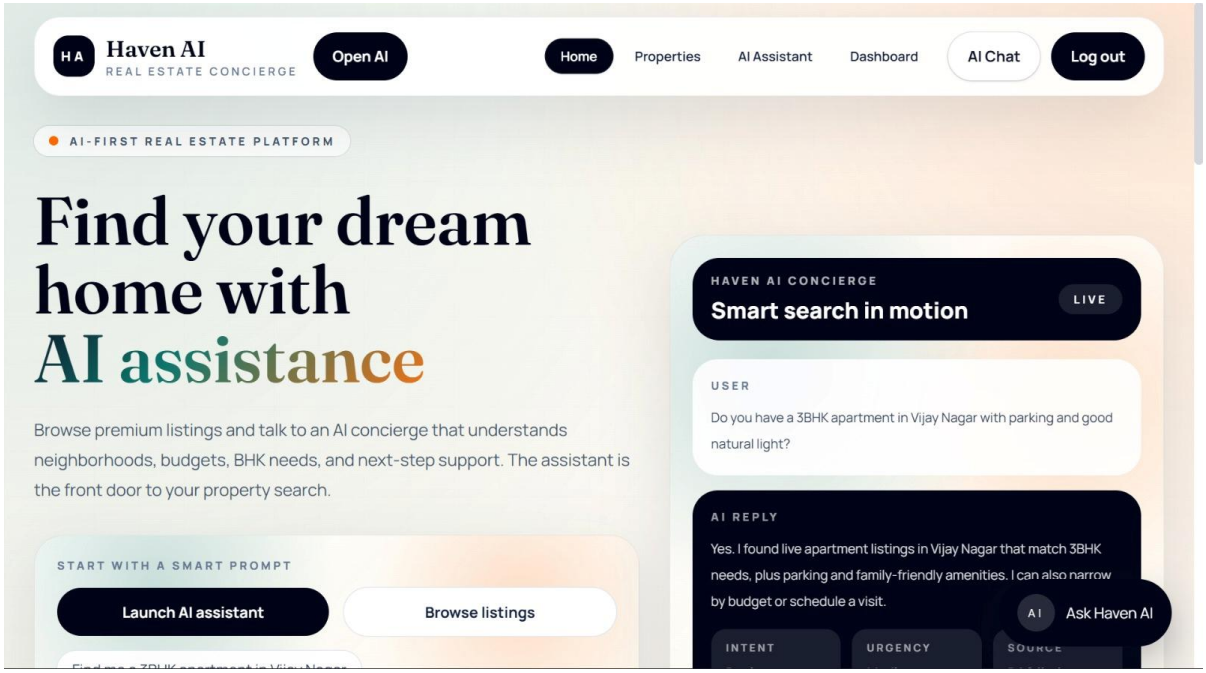
Architecture Diagram

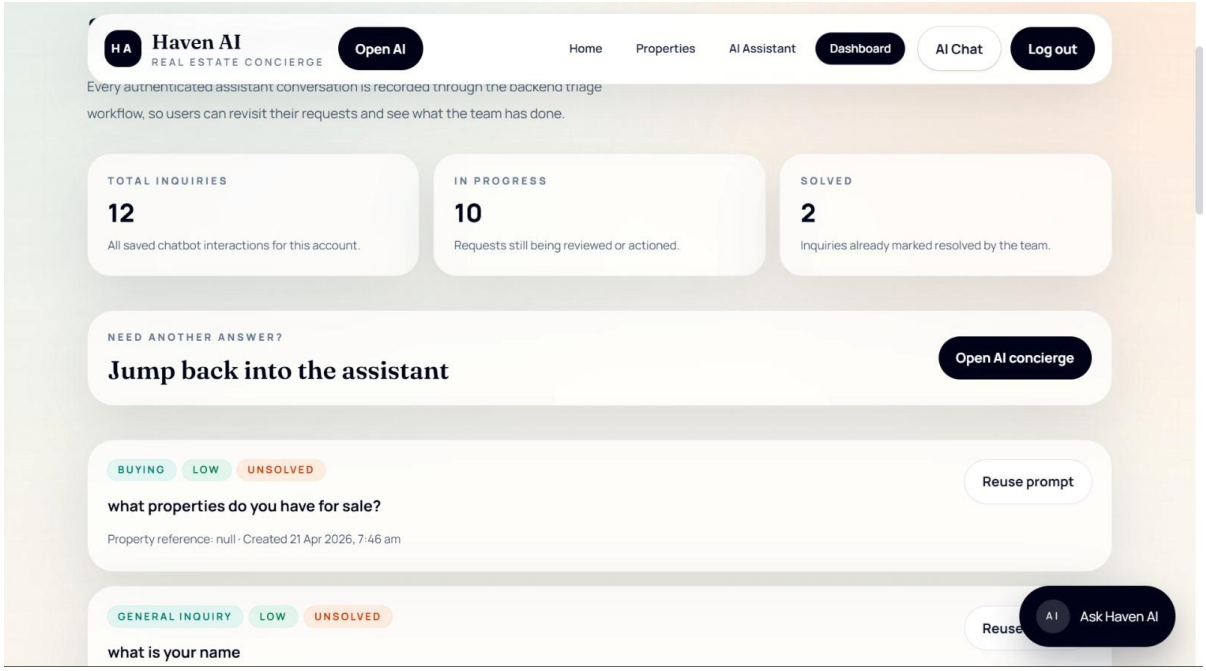
3. Results & Output

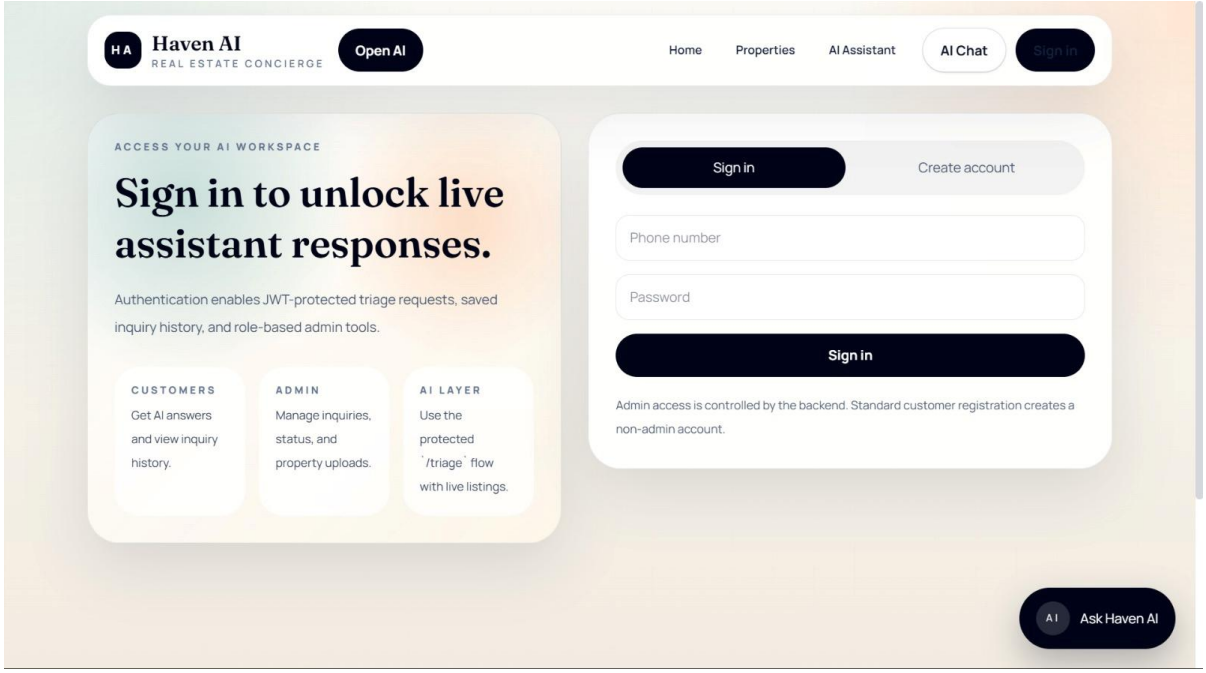
3.1 Screenshots / Outputs

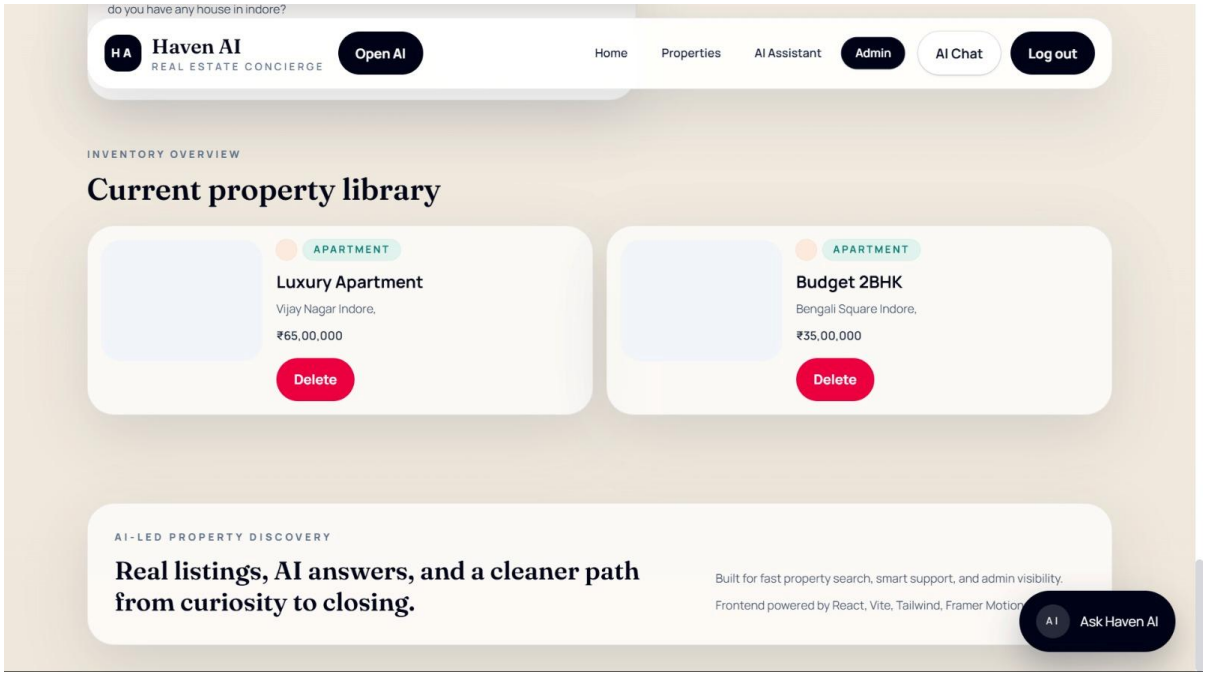
The developed system demonstrates the successful integration of full-stack AI development and cloud automation:

- **Property Browsing Interface:** A dynamic React-based UI displaying real estate listings fetched from the PostgreSQL database, featuring image galleries and property details.
- **AI Chat Output:** The chatbot interface displays conversational responses. When a user asks "Find a 3BHK under $500k," the AI responds with specific properties that actually exist in the database, demonstrating the successful execution of the RAG pipeline and the elimination of AI hallucinations.
- **Jenkins Pipeline Execution:** The Jenkins dashboard shows successful automated builds, visually indicating that code pushed to GitHub triggered a successful Docker image creation and registry push.
- **AWS & Infrastructure Status:** The AWS console reflects the active EC2 instances provisioned entirely via Terraform scripts, ensuring consistent and reproducible infrastructure.




HA
Haven AI
REAL ESTATE CONCIERGE
Open AI
Home
Properties
AI Assistant
Dashboard
AI Chat
Log out
workflow, so users can revisit their requests and see what the team has done.
TOTAL INQUIRIES
12
All saved chatbot interactions for this account.
IN PROGRESS
10
Requests still being reviewed or actioned.
SOLVED
2
Inquiries already marked resolved by the team.
NEED ANOTHER ANSWER?
Jump back into the assistant
Open AI concierge
BUYING
LOW
UNSOLVED
Reuse prompt
what properties do you have for sale?
Property reference: null · Created 21 Apr 2026, 7:46 am
GENERAL INQUIRY
LOW
UNSOLVED
what is your name
AI
Ask Haven AI


HA
Haven AI
REAL ESTATE CONCIERGE
Open AI
Home
Properties
AI Assistant
AI Chat
Sign in
ACCESS YOUR AI WORKSPACE
Sign in to unlock live assistant responses.
Authentication enables JWT-protected triage requests, saved inquiry history, and role-based admin tools.
CUSTOMERS
Get AI answers and view inquiry history.
ADMIN
Manage inquiries, status, and property uploads.
AI LAYER
Use the protected `/triage` flow with live listings.
Sign in
Create account
Phone number
Password
Sign in
Admin access is controlled by the backend. Standard customer registration creates a non-admin account.
AI
Ask Haven AI



3.2 Dashboards

The system provides multiple layers of monitoring and management:

- **Administrative Property Dashboard:** Administrators have access to a gated React interface where they can add, update, or remove property listings. Any changes made here are automatically synced to both the PostgreSQL database and vectorized into ChromaDB.
- **Kubernetes/DevOps Dashboards:** Through the terminal and Kubernetes dashboard, developers can monitor the health of the application Pods, track the load balancer status, and verify that the frontend and backend services are running seamlessly across the cluster.

3.3 Key Outcomes

The implementation of the AI-Powered Real Estate Platform produced several highly impactful outcomes:

- **Factual AI Assistance:** Successfully implemented a RAG pipeline that prevents the AI from generating fake properties, drastically improving trust and reliability.
- **Accelerated Deployment:** The time required to set up a new production environment was reduced from hours of manual configuration to minutes by executing Terraform and Ansible scripts.

- **Scalable Architecture:** Transitioning from a monolithic structure to Dockerized microservices managed by Kubernetes ensures the application can handle varying loads of traffic.
- **Seamless Development Lifecycle:** The Jenkins CI/CD pipeline ensures that new features and bug fixes are deployed to production securely and automatically, reducing human error.

4. Conclusion

The AI-Powered Real Estate Platform successfully demonstrates the powerful intersection of Generative AI and modern DevOps practices. By integrating a Retrieval-Augmented Generation (RAG) architecture using Google Gemini and ChromaDB, the system provides users with an intelligent, conversational search experience that is strictly grounded in factual property data, solving the industry-wide problem of AI hallucination.

Furthermore, the project highlights the critical importance of robust infrastructure automation. By utilizing Terraform, Ansible, Docker, Kubernetes, and Jenkins, the platform moves beyond being a simple code repository to becoming a fully automated, scalable, cloud-native enterprise application. This ensures that the system is not only intelligent but also highly available, easily maintainable, and ready for rapid iteration.

5. Future Scope & Enhancements

While the current platform successfully implements RAG and automated deployments, several enhancements can be integrated into future iterations to elevate the system to an enterprise-grade level.

A primary enhancement would be the integration of **Advanced Monitoring and Observability** tools such as Prometheus and Grafana. This would provide real-time metrics on AI response latency, database query times, and Kubernetes cluster health, allowing for proactive system maintenance.

The AI capabilities can be expanded by incorporating **Multimodal RAG**. By allowing the AI to process images, users could upload a photo of a desired architectural style, and the system would perform a visual similarity search alongside semantic text search.

Additionally, the DevOps pipeline can be enhanced with **Automated Security Scanning (DevSecOps)**. Integrating tools like SonarQube for code quality and Trivy for Docker image vulnerability scanning within the Jenkins pipeline would ensure that security is enforced at every stage of the development lifecycle. Finally, migrating the database from a single instance to a distributed, highly available cluster would ensure maximum uptime during large-scale traffic events.